Inheritance

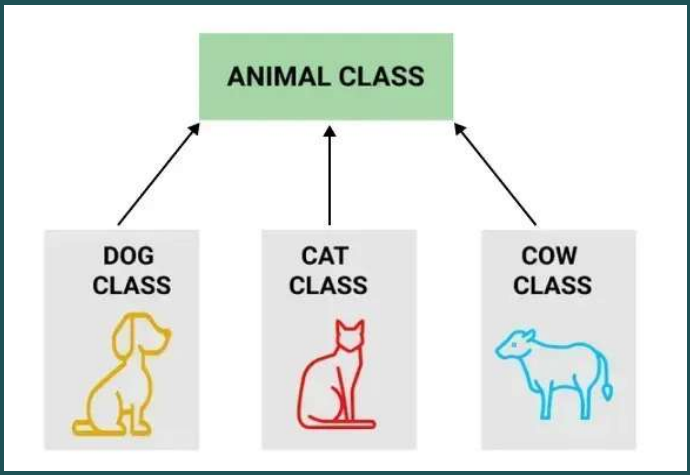
CHRISTIEN,
DAYENNE,
HIDDE
NICK,
SJORS,
YUNUS,
ZAKARIA

Inleiding


Wat betekent het in C++?
Hoe gebruik je het in echte projecten?
Wat zijn de voordelen en nadelen?
Wat zijn goede voorbeelden?
Waarom geen compositie maar inheritance?

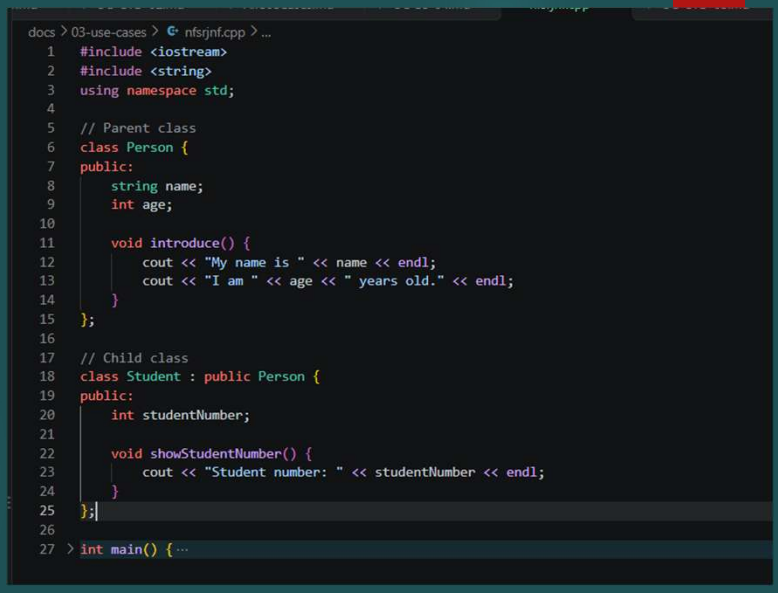
Wat betekent het in C++?

- Parent-child
- OOP



Wat zijn goede voorbeelden?

Parent class: Person
Child class: Student
: public Person
Overerving
Name & Age
StudentNumber
Codehergebruik
Minder duplicatie
Eigen functies toevoegen



Yunus

Wat zijn goede voorbeelden?

```
PS C:\Users\nabeu\Downloads> g++ yunustest.cpp
PS C:\Users\nabeu\Downloads> ./a.exe
My name is Yunus
I am 18 years old.
Student number: 12345
```

```cpp
int main() {

    // Create Student object
    Student student1;

    // Properties inherited from Person
    student1.name = "Yunus";
    student1.age = 18;

    // Own property of Student
    student1.studentNumber = 12345;

    // Methods inherited from Person
    student1.introduce();

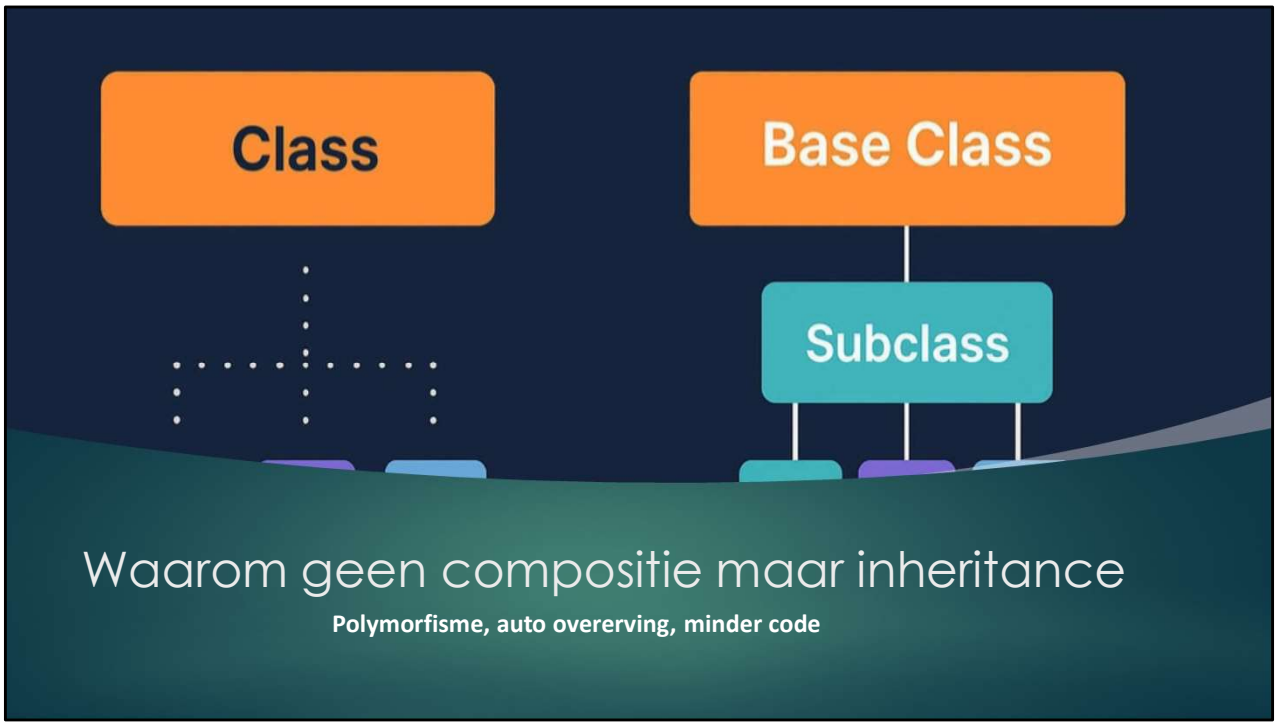
    // Own method of Student
    student1.showStudentNumber();

    return 0;
}
```

Yunus



Nick
Het voordeel aan inherintance is dat je een clas permanent "onderdeel" uit laat maken van een andere class, hierdoor bouw je een duidelijke structuur op die consisten blijft tijdens gebruik. Daarnaast kun je hiargiesch structuur maken, waarbij je makkelijk op een rij kan zetten wat eerst komt dus voertuig -> bus -> model 1682 -> stoelnummer.
Er bestaat echter wel een negatief probleem wat vaak voorkomt; THE DIAMOND PROBLEM, want wat voor stuur moet er nu in de autobus



Nick

- Minder code: je hoeft bestaande eigenschappen en methoden niet opnieuw te implementeren.
- Polymorfisme: je kunt verschillende child classes via één parent class behandelen.
- Automatische overerving: child classes krijgen direct toegang tot de publieke en protected functionaliteit van de parent.
- Logische structuur: je kunt een algemene klasse maken waar specifiekere klassen van afgeleid zijn.